

BRL Architecture

CHANGMIN PARK* and SEONGAH PARK*, Sogang University, Korea

NAYOUNG KIM, Sogang University, Korea

SUNGJIN PARK, Sogang University, Korea

GEONYEONG KO, Sogang University, Korea

JUNHO LEE, Sogang University, Korea

CHANGJOO NAM, Sogang University, Korea

In this paper, we propose a robotic system that integrates symbolic planning, uncertainty modeling, and user corrective feedback in a knowledge-based framework to reduce human intervention and workload. Existing human-in-the-loop robot systems often require continuous user involvement, which increases interaction frequency and workload. To address this challenge, we model the system as a Partially Observable Markov Decision Process (POMDP) and track uncertainty during planning and execution. We introduce an entropy-based quantitative measure to evaluate uncertainty and generate queries that focus on the sources of ambiguity. The system requests user feedback if the estimated uncertainty exceeds a threshold and updates the knowledge base accordingly. The proposed system is integrated into a real robotic platform. Experimental results show that it maintains a comparable task success rate to systems that rely on frequent user intervention, while reducing the number of queries.

CCS Concepts: • **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; *Generate the Correct Terms for Your Paper*; *Generate the Correct Terms for Your Paper*.

Additional Key Words and Phrases: robotic system, planning under uncertainty, human-robot interaction

ACM Reference Format:

Changmin Park, Seongah Park, Nayoung Kim, Sungjin Park, Geonyeong Ko, Junho Lee, and Changjoo Nam. 2026. BRL Architecture. In *Proceedings of Transactions on Human-Robot Interaction (Conference acronym 'XX)*. ACM, New York, NY, USA, ?? pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Recent advances in robotic systems have significantly improved autonomy in perception, manipulation, and planning across domains such as agriculture and household environments. Despite these advances, robots still frequently fail in real-world environments due to perception errors [?], action execution failures [?], and dynamic environmental changes [?]. Consequently, many robotic systems still rely on human operators to continuously monitor execution and intervene when failures occur [?]. However, requiring humans to constantly observe robotic systems and manually

*Both authors contributed equally to this research.

Authors' Contact Information: Changmin Park, cmp9877@gmail.com; Seongah Park, ppseongah@gmail.com, Sogang University, Seoul, Korea; Nayoung Kim, Sogang University, Seoul, Korea, larst@affiliation.org; Sungjin Park, Sogang University, Seoul, Korea; Geonyeong Ko, Sogang University, Seoul, Korea; Junho Lee, Sogang University, Seoul, Korea; Changjoo Nam, Sogang University, Seoul, Korea, cpalmer@prl.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

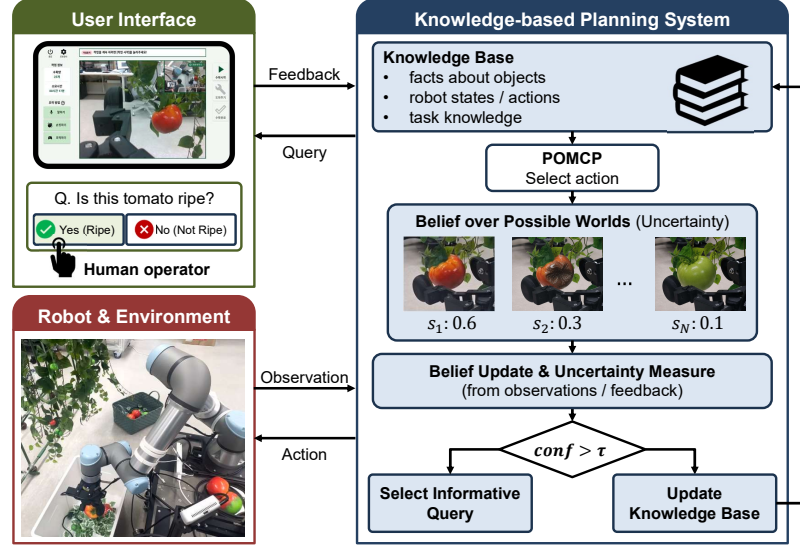


Fig. 1. Overview of the proposed knowledge-based robotic system with uncertainty-aware planning. The system maintains a belief over possible worlds and updates it based on observations. If the uncertainty exceeds a threshold, the system selectively queries the user to resolve ambiguity. Otherwise, it continues autonomous planning and execution.

determine whether the robot is making reliable decisions is impractical for long horizon human–robot collaboration. Continuous monitoring increases cognitive workload and fatigue [?], particularly in real-world deployments where robots operate for extended periods of time. Instead of relying on humans to identify failures after they occur, robotic systems should be able to recognize uncertain situations on their own and proactively request assistance only when necessary [? ?]. This raises two important challenges for collaborative robotic systems, namely (i) determining *when* uncertainty becomes significant enough to require human intervention and (ii) deciding *what* information should be queried to effectively resolve the uncertainty.

To address these challenges, we formulate robotic decision making as a POMDP [?] and develop a framework that jointly models symbolic knowledge [?] and uncertainty within a unified structure. We adopt a STRIPS-style symbolic representation [?], where the knowledge base is defined as a set of explicit facts. In addition, the system maintains a belief distribution over possible world states, which represent alternative hypotheses about the environment. As observations are collected, the system updates the belief over these states accordingly. **By measuring the entropy of the belief distribution, the system estimates uncertainty over possible worlds** and determines *when* to query a human. The system also determines *what* to ask by selecting questions that are expected to maximally reduce the remaining uncertainty between competing world states. Since the interaction only requires simple feedback about the environment state, the proposed framework reduces user cognitive burden without requiring users to make planning decisions.

We design a system architecture for real-world robotic operation based on this framework, as illustrated in Fig. ??. The system performs decision making using a POMDP formulation, maintaining a belief over possible worlds and updating it based on observations. A centralized knowledge base integrates sensing, planning, feedback management, and user interface modules. Sensor observations and user feedback directly update the knowledge base and are shared across modules. **This design maintains a consistent representation of information and ensures that planning uses the most up-to-date state.**

Table 1. Qualitative comparison of related works.

Related Works	Symbolic KB	Symbolic Planning	Planning under Uncertainty	Human Feedback	Proactive Query
Logic-based knowledge inference [????????]	○	×	△	×	×
Classical planning [?????]	○	○	×	×	×
LLM-based knowledge inference [??????]	○	△	×	×	×
LLM-based planning [????????]	○	△	△	△	×
POMDP-based planning [????????]	×	△	○	×	×
Symbolic planning under uncertainty [????]	○	○	○	×	×
Human-on-the-loop systems [????]	×	×	△	○	×
Uncertainty-aware query systems [???]	△	×	○	○	○
Ours	○	○	○	○	○

We evaluate the proposed approach in two domains, tomato harvesting and waste sorting. We demonstrate that our method reduces the number of interactions while maintaining or improving task success rates. In addition, user studies confirm that the proposed approach effectively reduces cognitive load during human–robot interaction.

The main contributions of this work are as follows:

- We propose a knowledge-based POMDP framework that explicitly represents uncertainty over symbolic world states for robotic decision making under partial observability.
- We introduce an entropy-based uncertainty estimation method that enables the robot to determine when human feedback is required during execution.
- We propose a query selection method that identifies informative environment states to determine what to ask in order to reduce uncertainty with minimal user interaction.
- We validate the proposed framework through user studies and show that it reduces user workload during human–robot collaboration.

2 Related Work

In this section, we review related work along three directions. We first discuss knowledge-based robotic systems for structured environment representation. We then review planning under uncertainty, including POMDP-based and belief space methods. Finally, we examine human corrective systems that incorporate feedback for decision making.

2.1 Knowledge-based Robotic System

In robotics, a knowledge base refers to a structured representation of information required for decision making. These systems use logic-based representations of objects, attributes, and relations to describe the world in a way that is understandable not only by robots but also by users [? ?]. In particular, predicate-based symbolic representations such as [? ?] allow the system to describe both object properties and relationships among objects. In addition, graph-based

models such as [?] are often used to represent these relationships more clearly by organizing objects and their connections in a simple structured form.

To systematically define robot states and actions, planning languages such as PDDL [?] or FDR [?] are widely adopted. In these representations, a state is defined as a set of predicates, and each action is described by its preconditions and effects, which specify how the state changes after execution. This formulation provides a clear model of state transitions and allows robots to plan over a well-defined symbolic state space. As a result, classical planning techniques [?] can be applied to compute action sequences that achieve a given goal.

Knowledge-based robotic systems store observed facts and infer new knowledge from existing information within this symbolic representation. Logical and relational inference allows robots to identify facts that are not directly observed, combine multiple relations, and obtain plausible hidden states from contextual knowledge [?]. For example, a robot can infer the likely location of an unseen object from qualitative spatial relations, container-location relations, or place-object co-occurrence knowledge [?]. Commonsense reasoning connects symbolic knowledge with perceptual evidence and supports decision making in partially observable environments [?]. Ontology structures and type hierarchies further support consistent integration of new observations and generalization across related objects, actions, and task contexts [?].

While symbolic knowledge-based systems provide structured representations and support logical inference, recent approaches have explored the use of large language models (LLMs) to further expand how knowledge is utilized in robotic planning. Early LLM-based approaches employ procedural and commonsense knowledge encoded in LLMs to propose task decompositions or action sequences. The proposed plans are then grounded with robot affordances, programmatic interfaces, or execution feedback [?]. Recent LLM-based approaches also incorporate external knowledge sources to compensate for incomplete observations. Some methods use scene graphs or open-vocabulary spatial representations to provide structured information about objects, places, and relations [?]. Other methods retrieve demonstrations, procedural knowledge, or task-relevant constraints to guide long-horizon planning [?]. Related methods combine LLM-generated plans with symbolic constraints, spatial reasoning, or verification mechanisms to improve executability and consistency [?].

However, many knowledge-based robotic systems still rely on deterministic symbolic representations and assume a single consistent world state, which limits their ability to represent multiple hypotheses under uncertainty [?]. As a result, such systems do not explicitly model uncertainty in action effects, sensing, or world states, and inferred facts are typically committed as definite knowledge. Recent LLM-based planning methods also suffer from grounding errors, incomplete retrieval, and hallucinated inferences, and do not maintain structured uncertainty over alternative states. While knowledge-based systems provide rich semantic structure for planning, they do not directly address uncertainty in observations, inferred facts, and knowledge validity.

To address this limitation, we maintain multiple symbolic hypotheses over possible worlds instead of committing inferred facts as deterministic knowledge. Our framework combines symbolic knowledge-based reasoning with planning under uncertainty, allowing the robot to reason over uncertain inferred states while selectively requesting human feedback if the uncertainty becomes high.

2.2 Planning under Uncertainty

2.2.1 POMDP.

Unlike deterministic symbolic knowledge-based systems that assume a single world state, POMDPs explicitly reason over uncertainty through belief representations over multiple possible states. In robotic environments,

such uncertainty naturally arises from sensor noise, occlusions, incomplete observations, and unexpected state changes. Sequential decision-making under these conditions has been extensively studied within the framework of POMDPs [? ? ? ?].

A key distinction between a POMDP and an MDP is the introduction of a belief state, which represents a probabilistic distribution over possible states when the true state cannot be directly observed. Accordingly, a wide range of solution methods have been proposed, including point-based value iteration [?], online tree search [?], and Monte Carlo tree search (MCTS) [?].

Despite these advances, effectively representing and updating the belief state remains a fundamental challenge. Since a belief is inherently a continuous distribution over the state space, exact inference is computationally difficult. Moreover, as the state space grows, the belief space expands combinatorially, making inference increasingly intractable. As a result, various approaches have been proposed to approximate the belief state.

2.2.2 Belief Approximation. Studies on belief approximation can be broadly categorized into particle-based and factored-based. Particle-based methods approximate the belief as a set of sampled states and have been widely adopted due to their natural compatibility with sampling-based online planning [? ?]. For example, [?] maintains a set of particles as a history-conditioned belief and estimates action values by sampling future trajectories from a generative model without performing exact Bayesian belief updates. Similar sampling-based ideas have also been extended to large and continuous POMDPs, where DESPOT [?] evaluates a limited set of sampled scenarios and POMCPOW [?] controls search tree growth in continuous action and observation spaces.

In robotic domains, particle-based belief representations are widely used in localization, navigation, SLAM, object search, and manipulation, where particles encode hypotheses over robot, map, and object states under uncertainty. In navigation, [?] combines POMCP with model predictive control to maximize information gain over occupancy-grid maps, while [?] uses hierarchical local online POMDP planning for exploration in large unknown environments. Similar representations have also been used for continuous control under uncertainty, including autonomous driving at occluded intersections [?] and UAV collision avoidance or target tracking under partial observability [? ?]. In object search and manipulation, particles often represent complete scene hypotheses. For example, [?] samples visible and occluded object configurations for cluttered object search, while [? ?] uses particle-based scene hypotheses for multi-object manipulation. Collectively, these studies show that particle beliefs provide a flexible approximation for preserving multimodal dependencies among robot, environment, and object states.

In contrast, factored belief representations model the belief by decomposing the state into a set of variables and exploiting their structure [? ? ?]. Such approaches have been widely used in structured probabilistic models, including dynamic Bayesian networks and Hidden Markov Models [?], where the state is represented in a factored form. To improve computational efficiency, various approximate inference methods have been proposed to reduce the complexity of belief updates by relaxing dependencies among variables. For example, [?] approximates belief updates by maintaining a factorized representation while partially accounting for dependencies between variables. While factored representations exploit internal structure of state space, their independence assumptions break in symbolic planning where facts are tightly linked by action constraints. Therefore, we use a particle-based approximation to preserve these dependencies.

2.2.3 Symbolic Planning under Uncertainty. Symbolic planning formalisms such as PDDL [?], STRIPS [?], and FDR [?] have been widely used due to their interpretability [?]. These representations are typically based on first-order logic or similar discrete formalisms [?] which describes planning problems through explicit constraints and state transition rules. However, since traditional predicate logic only allows binary true or false values, it is difficult to directly model observation noise or state uncertainty. To address this limitation, various approaches have attempted to

integrate probabilistic reasoning into logical representations. For example, [?] assign weights to logical rules to enable probabilistic inference over uncertain relationships, and [?] combine logical structure with probabilistic reasoning.

However, these approaches primarily focus on building inference models, and their high cost in model construction is unsuitable for online decision-making with complex constraints.

As a result, several studies have explored planning under uncertainty while preserving symbolic structure [? ?]. For example, [?] encodes commonsense knowledge in ASP [?] to construct a prior over possible worlds and integrates it with probabilistic planning [?]. However, since the belief is defined over a set of possible worlds, the computational cost increases as the number of states grows. To address this, some works organize possible worlds in a structured manner. For instance, [?] construct a state space tree and a hierarchy of POMDPs based on domain knowledge, reducing complexity by leveraging abstraction. More recently, large language models have been used for belief approximation, as in [?], which builds a tree of hypotheses and uses LLMs to generate particles in open-ended settings. While these approaches enable more flexible or scalable belief representations, they either depend on predefined hierarchical structures or rely on the quality and consistency of generated hypotheses.

Prior work has explored various belief representations, but global belief approaches remain sample-inefficient under strong symbolic constraints. We instead approximate belief locally by sampling states reachable under each action [?]. These local beliefs are organized in a tree structure, and planning is performed using the off-the-shelf planner POMCP within an interpretable symbolic space. However, uncertainty in real-world robotic systems cannot always be resolved through planning and observation alone. Robots may therefore require additional information to disambiguate conflicting hypotheses, which motivates human corrective systems that selectively request feedback under high uncertainty.

2.3 Human Corrective System

Despite recent advances in robotic systems, robots still face operational uncertainty in dynamic and unpredictable real-world environments. Prior work has addressed this limitation by involving humans in different forms of robot autonomy [? ?]. In particular, in human-on-the-loop (HOTL) systems, robots execute tasks autonomously, while human operators monitor the system and revise or override robot decisions if failures or errors occur [? ?]. This supervisory structure can improve safety and robustness because human operators can compensate for errors that autonomous systems cannot resolve.

However, HOTL systems require operators to sustain attention and detect failures during robot operation. To provide effective corrective feedback, operators must first notice that the robot is in an uncertain or erroneous state. Situation awareness research suggests that operators must then understand the current situation, project possible consequences, and determine an appropriate intervention [?]. This cognitive process can be difficult in repetitive tasks. Operators may experience vigilance decrement because continuous exposure to monotonous monitoring tasks can slow responses and reduce detection accuracy [? ?]. Operators may also experience inattention blindness, in which visible failures are missed if attention is not directed to the relevant event [?]. As a result, continuous supervision can increase workload and fatigue [?]. Continuous supervision can also reduce the likelihood that operators detect critical robot failures [? ?].

Prior work has shown that robots can improve task performance, team efficiency, and subjective satisfaction if they proactively provide assistance during human-robot collaboration [?]. This finding suggests that proactive robot behavior can support collaboration by reducing the need for humans to manage every decision or detect every need for intervention. In parallel, uncertainty-aware robot systems have advanced toward determining when human input is needed. For example, conformal prediction-based methods construct prediction sets with statistical guarantees and use

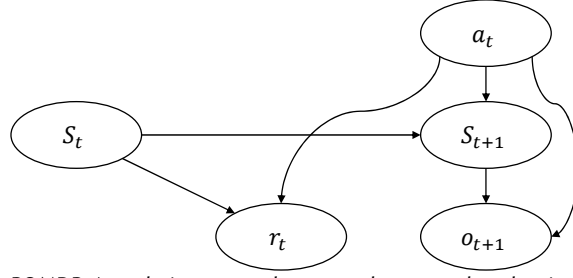


Fig. 2. Graphical illustration of a POMDP. At each time step t , the agent selects an a_t based on its current belief over the latent state s_t . The environment transitions to the next state s_{t+1} and generates a reward r_t . Since the true state is not directly observable, the agent receives an observation o_{t+1} .

the size of these sets to decide whether to query the user [? ? ?]. These methods show that robots can use uncertainty estimates to reduce unnecessary human intervention.

However, studies that develop and evaluate system-initiated query systems have mainly focused on technical performance, such as task performance, uncertainty calibration, and query efficiency. Human factors, such as workload and fatigue, can also affect task performance in automated systems that involve human operators [? ? ?]. Therefore, system-initiated corrective interaction should be evaluated not only in terms of technical performance but also in terms of operator burden and fatigue.

Building on this gap, we examine how system-initiated corrective interaction affect operator workload, fatigue, and task performance in automated robot systems. Our system does not require users to continuously supervise the robot to detect failures. Instead, users provide corrective feedback only if the system requests input. This interaction structure shifts part of the problem detection burden from the user to the robot.

3 Preliminaries

3.1 Symbolic State and Action Representation

The objective of planning is to find a transition sequence that transforms the initial state into a goal state. The planning problem is defined as a tuple $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, \mathcal{G} \rangle$, where $\mathcal{F}$ is a finite set of **atoms**, $\mathcal{A}$ is a set of actions, $s_0 \subseteq \mathcal{F}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{F}$ is the goal condition. A state s is represented as a subset of $\mathcal{F}$. Each action $a \in \mathcal{A}$ is defined as a tuple $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$, where $\text{pre}(a), \text{add}(a), \text{del}(a) \subseteq \mathcal{F}$ denote the preconditions, add lists, and delete lists of the action, respectively. An action a is said to be applicable in state s if and only if $\text{pre}(a) \subseteq s$, denoted as $a \llbracket s \rrbracket$. The state transition is defined as $s \xrightarrow{a} s'$, where $s' = (s \setminus \text{del}(a)) \cup \text{add}(a)$.

A plan is a sequence of actions $\pi = (a_1, \dots, a_T)$ such that the goal condition is satisfied after sequentially applying the actions starting from the initial state, i.e.,

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_T} s_T \quad \text{and} \quad \mathcal{G} \subseteq s_T.$$

3.2 POMDP

A POMDP is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and Ω is the observation space. The transition model $T(s' \mid s, a)$ specifies the probability of reaching the next state s' after

taking action a in current state s . The observation model $O(o \mid s', a)$ defines the probability of receiving observation o after arriving at state s' by executing action a . The reward function $R(s, a)$ gives a scalar value to each transition, and $\gamma \in (0, 1]$ is the discount factor.

Since the true state is not directly observable, the agent maintains a belief b , which is a probability distribution over $\mathcal{S}$ such that $b(s) = \Pr(s)$. After executing an action a and receiving an observation o , the belief is updated using a Bayes filter.

$$b'(s') = \eta O(o \mid s', a) \sum_{s \in \mathcal{S}} P(s' \mid s, a) b(s), \quad (1)$$

where η is a normalization constant.

A policy π maps beliefs to actions. The objective of a POMDP is to find a policy that maximizes the expected cumulative discounted reward, defined as $\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$.

3.3 POMCP

Partially Observable Monte Carlo Planning (POMCP) performs Monte Carlo tree search over action-observation histories $h = \langle a_1 o_1 \dots a_t o_t \rangle$, where each node in the search tree corresponds to a history. Action selection follows the UCB rule:

$$a^* = \arg \max_a \left(V(ha) + c \sqrt{\frac{\log N(h)}{N(ha)}} \right), \quad (2)$$

where $V(ha)$ is the estimated value of a after h , $N(h)$ and $N(ha)$ denote visit counts, and c is an exploration constant.

Each simulation samples the next state, observation, and reward from a generative model $(s', o, r) \sim \mathcal{G}(s, a)$. The return is computed recursively as $R = r + \gamma R'$, where R' is the return from subsequent steps. The value estimate is updated using an incremental average:

$$V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}. \quad (3)$$

The belief associated with a history h is represented implicitly as a set of particles, $B(h) = \{s \mid s \text{ is visited at } h\}$, which approximates the posterior over states without explicit belief updates.

4 System Architecture

In this section, we present an overview of the system architecture and the data flows between modules. Then we give a detailed explanation of their functionalities and applications.

4.1 Overview

Fig. ?? illustrates the brief system architecture and the data flow between the main modules. We integrate the *Knowledge Base* (KB), the *Planning Manager* (PM), the *Sensing and Perception* module (SP), the *Feedback Manager* (FM), and the *User Interface* (UI). The KB maintains symbolic descriptions and facts used for planning, SP generates sensory information from robot sensors, PM generates and executes plans, FM coordinates corrective feedback, and UI bridges the system and the user.

The KB stores essential information required for the robot to perform tasks and explicitly represents the current state of the system. All modules share information and interact with each other through the KB. The KB is composed of three types of information: domain knowledge, static information, and local information. Domain knowledge refers to prior knowledge in a specific domain. It includes descriptions of robot skills, domain-specific rules, and general explanations

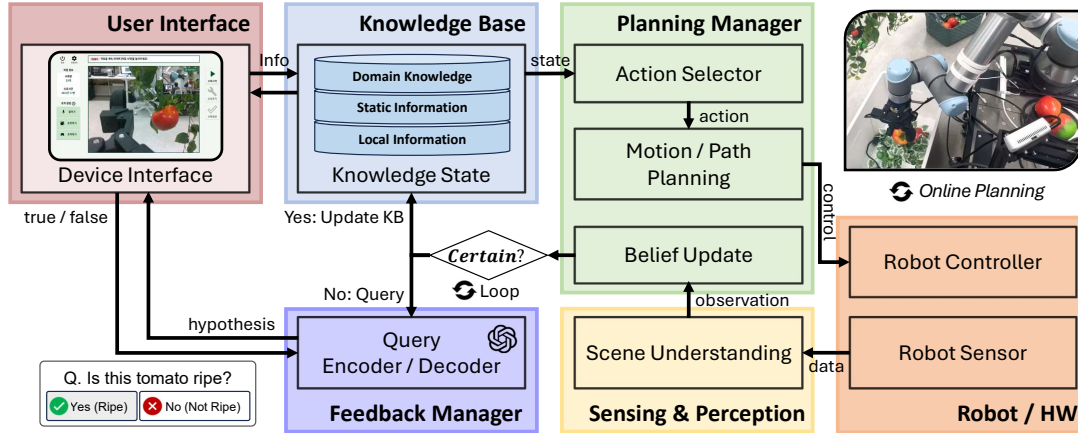


Fig. 3. System architecture and data flow of the proposed framework. The system consists of the *Knowledge Base* (KB), *Planning Manager* (PM), *Sensing and Perception* module (SP), *Feedback Manager* (FM), and *User Interface* (UI). The PM performs online planning based on the current knowledge state stored in the KB and updates the belief using observations from SP. The system evaluates the confidence of the updated belief. If the confidence exceeds a threshold, the most probable hypothesis is committed to the KB. Otherwise, the FM generates a query and requests corrective feedback from the user through the UI. The feedback is then used to refine the belief and continue planning.

of the environment. Static information is used during planning and execution but remains unchanged throughout task execution. Unlike domain knowledge, static information is encoded in a structured form and includes information such as robot parameters and local maps. In contrast, local information is used during planning and execution but cannot be known in advance. It is obtained through observations from the SP module and stored in the KB in the form of facts.

The PM is responsible for online action selection and execution under partial observability. The POMCP-based planner within the PM module selects the best action from the current belief and triggers the SP module to acquire additional observations. For instance, if the robot is positioned in front of a tomato stem, it naturally executes a detect action to infer the locations and ripeness states of tomatoes. To quantify uncertainty, the PM approximates the belief in the form of particle and computes the entropy over the resulting belief distribution. If such uncertainty is detected, the PM queries the FM to obtain additional information that can resolve the ambiguity of the observation. Upon receiving a response from the FM, the PM updates the belief accordingly and promotes the most probable belief state to a certain state in the knowledge base.

The SP module provides the current visual sensing information to the UI in real time. In addition, it updates the KB with observations triggered by the PM. In this process, the SP module makes use of data from various sensors, such as cameras, LiDAR, and tactile sensors, to extract information used for task execution, including object locations, states, and properties. For example, from RGBD images, the SP module infers visual attributes such as the position, ripeness, and state of a tomato, while tactile sensors are used to perceive contact-based properties such as object stiffness or softness. The high-level perceptual information extracted by the SP module is triggered by the BT and updated in the KB in the form of symbolic facts.

The FM module is implemented using an LLM-based interface that translates symbolic hypotheses from the PM into user-friendly natural language queries. For example, a hypothesis such as `ripe(tomato1)` is converted into a question like "Is this tomato ripe?". Along with the query, the module provides corresponding visual cues in the user interface, such as highlighting the referenced object with a bounding box. The FM module also interprets user interactions with

the interface (e.g., button clicks) and maps them to boolean feedback, which is then incorporated back into the system as validated truth assignments.

Finally, the UI displays the current visual sensing information and the system state, and allows the user to provide direct feedback or make selections. The user interacts with the FM module through the UI to monitor and supervise task execution.

4.2 Modules

4.2.1 Knowledge Base. The KB maintains a structured representation of the environment, including objects, properties, task-relevant relations, numeric quantities, and their associated confidence values. Formally, we define $\mathcal{KB} = \mathcal{I}^D \sqcup \mathcal{I}^S \sqcup \mathcal{I}^L$,¹ where $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ denote the domain, static, and local information sets, respectively, as described in Sec. ??.

For, $\mathcal{I}^S$ and $\mathcal{I}^L$, each information set contains both symbolic facts and numeric fluents. A symbolic fact is defined as $f = \text{pred}(\mathbf{x})$, where $\text{pred}(\cdot)$ denotes a predicate symbol and $\mathbf{x}$ denotes its arguments. A numeric fluent is defined as $v = \langle g(\mathbf{x}), v \rangle$, where $g(\mathbf{x})$ denotes a numeric function over objects and $v \in \mathbb{R}$ denotes its value. For example, symbolic facts can represent object categories and relations, such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, while numeric fluents can represent measurable quantities, such as $\text{pose_x}(\text{tomato}_1) = 0.12$ or $\text{distance}(\text{robot}, \text{stem}_1) = 2.5 \text{ (m)}$.

Specifically, the set $\mathcal{I}^S$ contains information that remains invariant throughout planning. By definition, for all $a \in \mathcal{A}$, $\text{Effect}(a) \cap \mathcal{I}^S = \emptyset$, where $\text{Effect}(a) = \text{add}(a) \cup \text{del}(a)$ denotes symbolic effects and can be extended to include numeric updates. These elements capture structural properties of the environment that are not altered by any action. For example, predicates such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, as well as fixed numeric quantities like $\text{loc_x}(\text{stem}_1) = 1.2$, remain unchanged regardless of the executed actions.

In contrast, $\mathcal{I}^L$ denotes local information obtained through the SP module and is not known a priori. This includes perceptual predicates, such as object existence or category estimates, as well as numeric fluents estimated from sensor data. For example, predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ may be inferred from observations, and numeric values such as $\text{distance}(\text{robot}, \text{stem}_1) = 2.3$ or $\text{pose_x}(\text{tomato}_1) = -0.5$ are updated as new sensor measurements become available.

4.2.2 Planning Manager. The PM performs decision making based on the current belief state while continuously updating the knowledge base during execution. Uncertainty is quantified using the entropy of the belief distribution. If the uncertainty exceeds a threshold, the planner requests additional verification from the FM. Otherwise, it commits the most probable hypothesis to the knowledge base and proceeds with action execution. In the following, we describe the belief approximation, the uncertainty estimation, the POMCP based search, and the overall planning procedure.

Belief representation. In partially observable environments, the true state cannot be directly observed, and thus the PM maintains a belief, which is a probability distribution over states. The belief at time t is defined as $b_t(s) = P(s_t = s \mid h_t)$, where $h_t = \{a_1, o_1, \dots, a_t, o_t\}$ denotes the history of actions and observations. However, in symbolic domains, the state space is large and subject to structural constraints, making exact belief tracking computationally intractable. Thus, we approximate the belief using a set of particles.

¹The notation $\sqcup$ denotes a disjoint union, indicating that $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ form a partition of $\mathcal{KB}$.

Formally, at time step t , the belief is represented as a finite set of particles and their associated weights:

$$\mathcal{B}_t = \{(s_i, w_i)\}_{i=1}^N, \quad \sum_{i=1}^N w_i = 1, \quad (4)$$

where each s_i denotes a possible world composed of a set of facts. The approximated belief distribution is then

$$b_t(s) \approx \sum_{i=1}^N w_i \delta(s_i = s). \quad (5)$$

In the symbolic domain, actions exhibit partially predictable effects due to their interpretable structure. For example, `pick(tomato1)` may result in grasp failure or selecting a different tomato, but it does not plausibly change the location of the robot. Based on this observation, we define a restricted set of reachable successor states, referred to as the *frontier*. For each s and a , the frontier is defined as

$$\text{Frontier}(s, a) = \{s_1^\star, s_2^\star, \dots, s_j^\star\}, \quad (6)$$

where each $s_k^\star$ denotes a reachable successor state.

In contrast to standard particle filtering, we assume a *knowledge* state s^K that represents the set of certain facts. Given a and s^K , we construct the belief by considering only the reachable states induced by s^K . That is, particles are defined exclusively over the frontier $\text{Frontier}(s^K, a)$, and the belief is represented as

$$\mathcal{B}_t = \{(s_k^\star, w_k)\}_{k=1}^j, \quad \sum_{k=1}^j w_k = 1, \quad (7)$$

where each $s_k^\star$ is a reachable successor state such that $s_k^\star \in \text{Frontier}(s^K, a)$. After executing a and receiving an observation o , the belief over the frontier is updated by adjusting the weight of each particle according to how well it explains the observation. Specifically,

$$\forall s_k^\star \in \text{Frontier}(s^K, a), \quad w'_k = w_k \cdot O(o \mid s_k^\star). \quad (8)$$

followed by normalization to obtain the *uncertainty* of the current state.

Uncertainty of the state. Given the updated belief over the frontier, uncertainty is quantified based on the distribution of particle weights. Intuitively, uncertainty increases when multiple candidate states have comparable likelihoods, and decreases when the belief is concentrated on a single dominant state. This uncertainty arises from *hypotheses*, which denote literals whose truth values are not yet determined.

The confidence of the current belief is defined as

$$\text{Conf}(\mathcal{B}) = 1 - \frac{\mathcal{H}(\mathcal{B})}{\log_2 j}, \quad (9)$$

where j denotes the number of particles. In the degenerate case where $j = 1$, the belief is fully concentrated on a single state and we define $\text{Conf}(\mathcal{B}) = 1$. Here, $\mathcal{H}(\mathcal{B})$ denotes the entropy of the normalized particle weights. Let $\tilde{w}'_k$ denote the normalized weights such that $\tilde{w}'_k = w'_k / \sum_{l=1}^j w'_l$. The entropy is computed as

$$\mathcal{H}(\mathcal{B}) = - \sum_{k=1}^j \tilde{w}'_k \log_2 \tilde{w}'_k. \quad (10)$$

This normalization ensures that the confidence lies in $[0, 1]$, where a uniform belief yields low confidence and a concentrated belief yields high confidence.

If the confidence exceeds a predefined threshold τ , the most probable state is selected as the updated knowledge, $s^* = \arg \max_{s_k^*} w'_k$, and the belief collapses to this state by discarding all remaining particles. Otherwise, the system identifies uncertainty-inducing hypotheses that differentiate candidate states.

The candidate hypotheses are constructed from literals that differ between the current knowledge state and the frontier states. Let C denote this set of candidate hypotheses. For each $c \in C$, the belief is partitioned into two subsets depending on whether c holds:

$$\mathcal{B}_c^+ = \{(s_k^*, w'_k) \mid c \in s_k^*\}, \quad \mathcal{B}_c^- = \{(s_k^*, w'_k) \mid c \notin s_k^*\}.$$

The system selects the hypothesis that is expected to reduce the uncertainty the most by minimizing the expected entropy after querying:

$$c^* = \arg \min_{c \in C} [P(c)\mathcal{H}(\mathcal{B}_c^+) + P(\neg c)\mathcal{H}(\mathcal{B}_c^-)], \quad (11)$$

where

$$P(c) = \sum_{s_k^*: c \in s_k^*} w'_k \quad \text{and} \quad P(\neg c) = \sum_{s_k^*: c \notin s_k^*} w'_k.$$

The selected hypothesis c^* is passed to the FM as a query. After receiving the answer, the belief is filtered by removing particles that contradict the feedback. If the answer confirms c^* , only particles containing c^* are retained. Otherwise, particles containing c^* are discarded:

$$\mathcal{B} \leftarrow \begin{cases} \{(s_k^*, w'_k) \mid c^* \in s_k^*\}, & \text{if } ans = \text{true}, \\ \{(s_k^*, w'_k) \mid c^* \notin s_k^*\}, & \text{if } ans = \text{false}. \end{cases} \quad (12)$$

The remaining weights are renormalized, and the confidence is recomputed. This process is repeated until the confidence exceeds τ or no informative hypothesis remains.

POMCP-based search. We employ POMCP [?] as the underlying planner for action selection. We use its sampling-based framework and focus on integrating symbolic representations with uncertainty-aware reasoning. Instead, as summarized in Alg. ?? to Alg. ??, we apply minimal adaptations to ensure consistency with the symbolic domain.

In Alg. ??, the search phase follows standard POMCP by repeatedly sampling a state and performing simulations. Each sampled state is passed to SIMULATE (line ??), and the accumulated values are used to select the final action. If the belief is empty, the search uses s^K , and at the root, the action is chosen based on value without UCB (line ??).

In the simulation phase, as described in Alg. ??, the recursive structure and return computation remain the same as in standard POMCP, but both action selection and node expansion are restricted to $\mathcal{A}(s)$ (line ??). That is, instead of considering the full action space, only actions applicable in state s are explored. If a new history h is encountered, the node is expanded only with actions in $\mathcal{A}(s)$ (line ??), and the value is initialized via a ROLLOUT. Here, T denotes the search tree, while $N(ha)$ and $V(ha)$ represent the visit count and value of action a , respectively. The next state, observation, and reward are then sampled from the generative model $(s', o, r) \sim \mathcal{G}(s, a)$ (line ??). As a result, the simulation remains stochastic but is confined to reachable symbolic transitions.

In Alg. ??, the same principle is applied to both action selection and rollout. The candidate set $C = \{a \mid a[s] \in \mathcal{A}, ha \in T\}$ includes actions that are both applicable in state s and present in the tree. If use_ucb is true, an action is selected according to the UCB criterion defined in Eq. ??. Otherwise, the action with the highest value is chosen. During rollout, actions are sampled uniformly from all applicable actions. The generative model $\mathcal{G}(s, a)$ then produces the next state, observation, and reward, while the action space is restricted by symbolic constraints.

Algorithm 1: POMCP Search with Applicable Actions

```

1 Input: belief  $b$ 
2 Output: selected action  $a \in \mathcal{A}$ 
3 procedure SEARCH( $b$ )
4    $h \leftarrow$  root history
5    $s^K \leftarrow \text{knowledge}(b)$ 
6   repeat
7     if  $B(h) \neq \emptyset$  then
8       sample  $s \sim B(h)$ 
9     else
10       $s \leftarrow s^K$ 
11     SIMULATE( $s, h, 0$ )
12   until TIMEOUT;
13    $a \leftarrow \text{SEARCH-BEST}(h, s^K, \text{false})$ 
14   return  $a$ 

```

Algorithm 2: POMCP Simulation with Applicable Actions

```

1 Input: state  $s$ , history  $h$ , depth  $depth$ 
2 Output: simulated return  $R$ 
3 procedure SIMULATE( $s, h, depth$ )
4   if  $\gamma^{depth} < \epsilon$  then
5     return 0
6   if there is no  $a[s] \in \mathcal{A}$  then
7      $N(h) \leftarrow N(h) + 1$ 
8     return 0
9   if  $h \notin T$  then
10    foreach  $a[s] \in \mathcal{A}$  do
11       $T, N(ha), V(ha) \leftarrow T \cup \{ha\}, 0, 0$ 
12       $N(h) \leftarrow N(h) + 1$ 
13    return ROLLOUT( $s, depth$ )
14    $a \leftarrow \text{SEARCH-BEST}(h, s, \text{true})$ 
15   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
16    $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', hao, depth + 1)$ 
17    $B(h) \leftarrow B(h) \cup \{s\}$ 
18    $N(h) \leftarrow N(h) + 1$ 
19    $N(ha) \leftarrow N(ha) + 1$ 
20    $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$ 
21   return  $R$ 

```

In symbolic domains, action preconditions and effects are explicitly defined, so exploring infeasible actions is unnecessary. Restricting the action space to $\mathcal{A}(s)$ therefore improves both search efficiency and model consistency. Overall, the structure follows POMCP while naturally incorporating the constraints imposed by the symbolic representation.

Algorithm 3: Action Selection and Rollout

```

1 Input: history  $h$ , state  $s$ 
2 procedure SEARCH-BEST( $h, s, use\_ucb$ )
3    $C \leftarrow \{a \mid a \llbracket s \rrbracket \in \mathcal{A}, ha \in T\}$ 
4   if  $C = \emptyset$  then
5     return  $\emptyset$ 
6   if  $use\_ucb$  then
7     return  $\arg \max_{a \in C} V(ha) + c \cdot \sqrt{\frac{\log N(h)}{1+N(ha)}}$ 
8   else
9     return  $\arg \max_{a \in C} V(ha)$ 
10 procedure ROLLOUT( $s, depth$ )
11   if ( $\gamma^{depth} < \epsilon \vee \gamma = 0$ )  $\wedge$   $depth > 0$  then
12     return 0
13   if there is no  $a \llbracket s \rrbracket \in \mathcal{A}$  then
14     return 0
15    $a \sim \text{Uniform}(\{a \mid a \llbracket s \rrbracket \in \mathcal{A}\})$ 
16   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
17   return  $r + \gamma \cdot \text{ROLLOUT}(s', depth + 1)$ 

```

Action execution. Each $a \in \mathcal{A}$ selected by the POMCP is associated with an executable robot skill. The symbolic action specifies the task-level intention and parameters, while the corresponding skill defines the motion or perception routine required for execution. For example, a symbolic action such as `navigate(robot,  $l_1, l_2$ )` is mapped to a navigation routine, and `pick(robot, tomato)` is mapped to a predefined manipulation trajectory.

In this work, $\forall a \in \mathcal{A}$ follows an encoded execution policy that maps a symbolic operator to a robot skill through an off-the-shelf motion planner, such as MoveIt [?]. The planner takes symbolic facts and numeric fluents stored in the KB as inputs. Numeric fluents, such as object poses, distances, and grasping parameters, define the parameters of motion generation and maintain consistency between high-level planning and low-level execution.

Specifically, after each action execution, the system receives an observation o from the environment and uses it for belief update. The system uses these observations to update both symbolic predicates and numeric fluents in the KB. This design separates task-level decision making from low-level motion planning. The KB provides a shared representation across these levels. As a result, the PM does not depend on a specific motion planner implementation.

Overall planning pipeline. We adopt a online planning framework as described in Algorithm ???. The planning process starts from the initial knowledge state s_0^K , which is used to initialize the belief (line ??). At each iteration, the planner first selects a_t by performing Monte Carlo tree search over the current belief $\mathcal{B}$ (line ??). If no applicable action exists, the process terminates. Otherwise, the selected action is executed and an observation o_{t+1} is received from the environment.

The belief is then updated using the observation model (line ??), and the confidence of the updated belief is computed (line ??). If the confidence is below the threshold τ , the planner queries the FM to reduce uncertainty. Specifically, it selects the most informative hypothesis, sends it to the FM, and filters out particles inconsistent with the answer

Algorithm 4: Planning Pipeline

```

1 Input: initial knowledge  $s_0^K$ , confidence threshold  $\tau$ 
2 Output: plan  $\pi$ 
3 procedure PLANNING( $s_0^K$ )
4    $knowledge(\mathcal{B}) \leftarrow s_0^K$ 
5    $t \leftarrow 0$ 
6   repeat
7      $a_t \leftarrow \text{SEARCH}(\mathcal{B})$ 
8     if  $a_t = \emptyset$  then
9       // There is no  $a[s]$ . (Dead-end)
10      break
11     execute  $a_t$  and receive observation  $o_{t+1}$ 
12      $\mathcal{B} \leftarrow \text{UPDATE}(\mathcal{B}, a_t, o_{t+1})$  (Eq. ??)
13      $c \leftarrow \text{CONF}(\mathcal{B})$  (Eq. ??)
14     while  $c < \tau$  do
15        $hypo^* \leftarrow \text{SELECT-HYPO}(\mathcal{B})$  (Eq. ??)
16        $ans \leftarrow \text{QUERY-FM}(hypo^*)$ 
17        $\mathcal{B} \leftarrow \text{FILTER}(\mathcal{B}, hypo^*, ans)$  (Eq. ??)
18        $c \leftarrow \text{CONF}(\mathcal{B})$ 
19      $s^K \leftarrow \arg \max_{s_k^* \in \mathcal{B}} w'_k$ 
20      $knowledge(\mathcal{B}) \leftarrow s^K$ 
21      $t \leftarrow t + 1$ 
22   until  $t = \text{MAX-STEP}$ ;
23   return  $\pi = (a_0, \dots)$ 

```

(lines ??-??). This process is repeated until the confidence exceeds τ . Once the belief becomes sufficiently confident, the most probable state is committed as the updated knowledge state, and the planner proceeds to the next step.

4.2.3 Sensing and Perception. The SP module acquires raw sensory data, generates local information in $\mathcal{I}^L$, and updates the numeric fluents in the KB. It also provides observations required for belief updates during action execution. Let $\mathcal{M}$ denote the set of perception models. For each predicate pred associated with local information in $\mathcal{I}^L$, there exists a corresponding perception model $m_{\text{pred}} \in \mathcal{M}$ defined as

$$m_{\text{pred}} : \mathcal{R} \rightarrow \mathcal{I}^L,$$

where $\mathcal{R}$ denotes the space of raw sensory data. Given a sensory input $r \in \mathcal{R}$, the model $m_{\text{pred}}(r)$ produces a symbolic fact $f = \text{pred}(\mathbf{x})$ or a numeric fluent $v = \langle g(\mathbf{x}), v \rangle$ in $\mathcal{I}^L$.

Through this process, perceptual predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ are instantiated, and numeric fluents such as $v = \langle \text{distance}(\text{robot}, \text{stem}_1), 2.3 \rangle$ or $v = \langle \text{pose_x}(\text{tomato}_1), -0.5 \rangle$ are continuously updated based on incoming observations. Since $\mathcal{I}^L$ represents information that is not known a priori, all elements in this set are generated and refined exclusively through the SP module.

In addition, the SP module defines the observation interface of the system. After executing an action $a \in \mathcal{A}$, the system receives an observation $o \in \mathcal{O}$ derived from the current sensory input, which is used to update the belief over

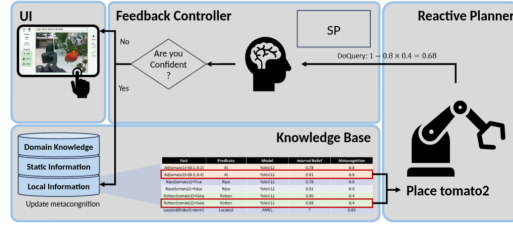


Fig. 4. An overview of FM system. This figure is a temporary image.



Fig. 5. An overview of interface. This figure is a temporary image.

possible states. Furthermore, if triggered by the PM, the SP module acquires new sensory input and evaluates the relevant perception models to produce observations and update the associated elements in $\mathcal{I}^L$.

4.2.4 Feedback Manager. As illustrated in Fig. ??, the FM translates hypotheses provided by the PM into a form that is easily understandable for the user, and interprets user responses into a format that can be processed by the system. Specifically, a hypothesis such as $\text{ripe}(\text{tomato}_1)$ is represented internally as a predicate, but the FM converts it into a natural question such as “Is this tomato ripe?”. To support user understanding, the queried object is visualized with a bounding box indicating its location.

In addition, the FM interprets user inputs. Users can respond via touch or speech, and the FM converts these responses into a boolean form (true or false) that can be used for belief updates. Through this bidirectional translation, the FM reduces the semantic gap between the system and the user, enabling efficient interaction.

4.2.5 User Interface. As shown in Fig. ??, the UI enables interaction between the user and the system and is implemented on a tablet. It presents queries generated by the FM and overlays the queried object on the camera image using a bounding box to support intuitive understanding. Users provide responses through simple touch input, with optional speech input. The interface hides internal representations and is designed to elicit accurate feedback with minimal workload.



Fig. 6. The illustration of the environments. (a) A UR5 robot sorts four types of waste into designated bins in Isaac Sim. (b) A mobile manipulator harvests ripe tomatoes and discards rotten ones in a real-world setting.

5 Experiments

We first evaluate the task success rate of the proposed system and analyze the effect of query frequency. If a query is triggered, we assume that the human always provides the correct answer. Through preliminary experiments, we found that $\tau = 0.8$ provides a reasonable trade-off between task performance and the number of queries. The success rates under different values of τ are reported in Appx. ?? . We then conduct a user study to evaluate how query frequency affects user workload. Finally, we conduct another user study using the same survey protocol to examine what to query. We compare our method with the KnowNo framework and analyze whether querying state-level information or action-level decisions results in lower user workload.

5.1 Experimental Setup

We evaluate the proposed method in two domains: a simulated waste sorting domain and a real-world tomato harvesting domain. The waste sorting domain is implemented in simulation using NVIDIA Isaac Sim, where a fixed UR5 robotic arm is used for pick and place task. Object detection is performed using the YOLOv12 model [?]. In contrast, the tomato harvesting domain is deployed on a real mobile manipulator consisting of a UR5e robotic arm mounted on a Summit mobile platform. The robot is equipped with an RGB-D camera for visual perception and a LiDAR sensor for localization, and performs object detection using the same YOLOv12.

5.1.1 Waste sorting domain. We consider a tabletop waste-sorting scenario with four waste objects placed on a table, as shown in Fig. ?? . A fixed UR5 robot is positioned between the objects and the recycling bins and performs three actions: detect, pick, and place. We assume that all objects are reachable by the manipulator.

Each object belongs to one of four categories: general, paper, can, and plastic. The object categories are not known a priori and must be inferred through perception. Since objects may be partially occluded by others, a single observation may not reveal all categories, requiring repeated detection during execution. The detect action can observe multiple objects simultaneously, after which the robot places each object into the corresponding recycling bin.

The initial arrangement of objects is randomized in every episode. A task is considered successful when all objects are correctly sorted within the maximum step limit. A reward of +5 is assigned when a waste object is placed into the recycling bin corresponding to its category, and an additional reward of +10 is given when all objects are correctly sorted. All other actions receive zero reward.

5.1.2 Tomato harvesting domain. We consider a tomato harvesting scenario in which four tomatoes are attached to two stems, denoted as stem_1 and stem_2, as shown in Fig. ?? . The robot navigates between three locations: dock_station, stem_1, and stem_2, and performs detect, pick, scan, place, and discard actions.

Each tomato can be in one of three states: ripe, unripe, or rotten. The tomato states are initially unknown and must be estimated through perception. During detection, ripe and rotten tomatoes are visually ambiguous and cannot always be reliably distinguished. To resolve this ambiguity, the robot performs a scan action after picking a tomato to verify its condition before harvesting or discarding it. Unripe tomatoes are left untouched.

The tomato locations are randomized across stems in every episode. A task is considered successful when all ripe tomatoes are harvested and all rotten tomatoes are discarded within the maximum step limit. A reward of +5 is assigned when a ripe tomato is correctly harvested or a rotten tomato is correctly discarded after scanning. An additional reward of +10 is given when all task-relevant tomatoes are processed correctly. All other actions receive zero reward.

5.1.3 Implementation Details. In all experiments, the planner used a discount factor $\gamma = 0.95$, UCB exploration constant $c = 1.0$, maximum simulation depth $depth_{\max} = 20$, and TIMEOUT = 100 Monte Carlo simulations per planning step. Each episode was limited to 50 steps. The confidence threshold for triggering user feedback was set to $\tau = 0.8$, which was selected empirically based on preliminary experiments (see Appx. ??).

5.2 System Evaluation

We evaluate the proposed system in terms of task success rate and query efficiency under different querying strategies. In all experiments, if a query is triggered, we assume that the user always provides the correct answer in order to isolate the effect of the querying strategy itself. We compare four variants: **Ours**, **All**, **Random**, and **No**. The **Ours** method selectively queries the user if the confidence of the current belief falls below the threshold $\tau = 0.8$. The **All** baseline requests user feedback at every decision step, corresponding to full human supervision. The **Random** baseline randomly triggers queries with the same average query frequency as **Ours**, without considering the belief confidence. The **No** baseline performs fully autonomous execution without any user feedback.

For each domain, we measure the task success rate, the average number of queries per episode, and the query ratio, which denotes the percentage of execution steps that triggered user feedback. In addition, we analyze how selective querying affects belief refinement during execution.

Table ?? summarizes the experimental results. In both domains, **All** achieves the highest task success rate because all ambiguities are resolved through continuous human supervision. In contrast, **No** shows a substantial decrease in performance due to accumulated perception errors and unresolved uncertainty during planning and execution.

The proposed method achieves a task success rate close to **All** while significantly reducing the number of queries. In the waste sorting domain, **Ours** achieves a success rate of XX.X% while reducing the average number of queries from XX.X to XX.X. Similarly, in the tomato harvesting domain, **Ours** maintains comparable task performance while requiring substantially fewer user interactions. These results indicate that selective querying can effectively resolve critical uncertainty without relying on continuous user supervision.

In contrast, **Random** exhibits lower task success rates despite using a similar number of queries as **Ours**. This result suggests that the timing and content of queries are important, and that randomly requesting feedback does not reliably resolve task-relevant ambiguity.

The proposed system triggers queries only when the confidence of the current belief becomes sufficiently low. In practice, such situations occur when multiple candidate states explain the current observation with similar likelihoods. For example, in the tomato harvesting domain, ripe and rotten tomatoes may appear visually similar during distant observation, while in the waste sorting domain, partially occluded objects may lead to ambiguous category predictions.

When a query is triggered, the system selects the hypothesis expected to reduce the uncertainty the most and filters the belief based on the user response. As a result, the belief distribution becomes concentrated on a smaller

Table 2. System evaluation under different querying strategies. **All** queries the user at every decision step, **No** performs fully autonomous execution without queries, and **Ours** selectively queries the user based on belief confidence ($\tau = 0.8$). Query ratio denotes the percentage of execution steps that triggered user feedback.

Domain	Method	Task Success Rate (%)	Average Queries	Query Ratio (%)
Waste Sorting	All	98.0	12.4 ± 0.0	100.0
	No	71.5	0.0 ± 0.0	0.0
	Ours	94.5	3.1 ± 1.4	25.0
Tomato Harvesting	All	96.0	10.6 ± 0.0	100.0
	No	68.0	0.0 ± 0.0	0.0
	Ours	92.5	2.8 ± 1.1	26.4

set of candidate states, allowing the planner to continue execution with higher confidence. Across both domains, we observe that most task failures in the **No** baseline originate from unresolved ambiguity, whereas the proposed method successfully avoids many of these failures through selective user feedback.

Overall, the experimental results demonstrate that the proposed framework can maintain high task performance while substantially reducing the amount of required human feedback. Unlike the **All** baseline, the system does not rely on continuous supervision, and unlike **No**, it can recover from uncertain observations through interaction. These results suggest that confidence-based selective querying provides an effective balance between autonomy and human intervention in partially observable robotic tasks.

5.3 User Study

5.4 Discussion

6 Conclusion and Future Work

In this paper, we proposed a robotic decision-making framework that integrates symbolic knowledge and uncertainty, with the goal of reducing user workload. The proposed method maintains a belief over possible worlds within a POMDP-based planning framework and quantitatively models uncertainty through a probability distribution over these worlds. Based on this, the robot can assess its own uncertainty and selectively query the human only when necessary. This approach enables stable task execution without continuous supervision and minimizes unnecessary interactions, thereby reducing user intervention.

In particular, we leverage entropy-based confidence to determine when to issue queries and select the most informative hypothesis to construct questions, improving the efficiency of interaction. Experimental results show that the proposed method reduces the number of human interactions while maintaining or improving task success rates, and effectively lowers user workload. User studies further support this trend, demonstrating that a system which actively manages uncertainty and requests assistance only when needed can operate effectively in real-world settings.

For future work, several extensions can be considered. First, since the current query mechanism is based on symbolic hypotheses, it can be extended to a more intuitive natural language interface to improve user experience. In addition, incorporating more refined models of perception uncertainty would improve the accuracy of uncertainty estimation. Furthermore, this approach can be extended to multi-robot settings, where multiple agents share uncertainty and coordinate their queries collaboratively.

Appendix Contents

This appendix provides supplementary materials to support the main text.

- (1) Appendix ?? Experiments
 - (a) Appendix ?? Prompts for Object Detection and Naming
 - (b) Appendix ?? Prompts for Property Probing
 - (c) Appendix ?? Prompts for Predicate Generator
 - (d) Appendix ?? Prompts for Action Generator
 - (e) Appendix ?? Prompts for Initial State Descriptor
 - (f) Appendix ?? Prompts for Goal State Descriptor
 - (g) Appendix ?? Prompts for Task Planner
 - (h) Appendix ?? Prompts for Replanning
- (2) Appendix ?? Definitions of Shapes, Dimension, and Properties
 - (a) Appendix ?? Shapes
 - (b) Appendix ?? Dimensions
 - (c) Appendix ?? Properties
- (3) Appendix ?? Results of Task Planner and Plan Validator
 - (a) Appendix ?? Task Planning
 - (b) Appendix ?? Plan Validation
- (4) Appendix ?? Experiments
 - (a) Appendix ?? Instances
 - (b) Appendix ?? Property Probing Execution
 - (c) Appendix ?? Task Planning Demo
- (5) Appendix ?? Execution Layer Reproducibility
 - (a) Appendix ?? Overview of the PbD Layer
 - (b) Appendix ?? Mapping Symbolic Actions to Skills
 - (c) Appendix ?? Controller Configuration
 - (d) Appendix ?? Success Criteria
 - (e) Appendix ?? Abort and Safety Conditions
 - (f) Appendix ?? PbD Execution Pseudocode
 - (g) Appendix ?? Example Skill Trace

A Evaluation on τ

Hi! My name is τ !

B Prompt

Prompt for feedback manager (Waste Sorting)

System: You convert one symbolic waste-sorting fact into exactly one concise Korean yes/no question. You must preserve the predicate meaning exactly. Do not change the predicate into another property. Ask only the final question in Korean.

Message: Instruction: Convert the given symbolic fact into a polite and unambiguous Korean yes/no question.

Important rules:

1. First identify the predicate name in the fact.
2. The question must ask about that predicate only.
3. Do not infer or replace it with another predicate.
4. If the fact is detected(W), ask whether the indicated waste item has been detected.
5. If the fact is plastic(W), can(W), paper(W), or general(W), ask about the indicated waste item's type.
6. If the fact is holding(R, W), ask whether the robot is holding the indicated waste item.
7. If the fact is handempty(R), ask whether the robot's hand is empty.
8. If the fact is in_bin(W, B), ask whether the indicated waste item is in the indicated bin.
9. The respondent may not recognize identifiers such as waste1, waste2, robot1, bin_01, or bin_02.
10. Assume the respondent sees only a tablet display where the relevant object, robot, or bin is visually indicated.
11. Output only the question.

Examples:

Fact: detected(waste1)

Question: 화면에 표시된 폐기물이 감지된 것이 맞나요?

Fact: plastic(waste1)

Question: 화면에 표시된 폐기물이 플라스틱인 것이 맞나요?

Fact: holding(robot1,waste2)

Question: 현재 로봇이 폐기물을 들고 있는 것이 맞나요?

Fact: in_bin(waste1,bin_01)

Question: 화면에 표시된 폐기물이 표시된 수거함 안에 있는 것이 맞나요?

Action: {selected action}

Fact: {selected hypothesis}

Predicate meanings:

- detected(W): waste item W has been detected.
- plastic(W): waste item W is plastic.
- can(W): waste item W is a can.

- paper(W): waste item W is paper.
- general(W): waste item W is general waste.
- holding(R, W): robot R is holding waste item W.
- handempty(R): robot R's hand is empty.
- in_bin(W, B): waste item W is in bin B.

Question:

Answer: 화면에 표시된 폐기물이 감지된 것이 맞나요?

Prompt for feedback manager (Tomato Harvesting)

System: You convert one symbolic tomato-harvesting fact into exactly one concise Korean yes/no question. You must preserve the predicate meaning exactly. Do not change the predicate into another property. Ask only the final question in Korean.

Message: Instruction: Convert the given symbolic fact into a polite and unambiguous Korean yes/no question.

Important rules:

1. First identify the predicate name in the fact.
2. The question must ask about that predicate only.
3. Do not infer or replace it with another predicate.
4. If the fact is located(R, L), ask whether the robot is at the indicated location.
5. If the fact is ripe(T), unripe(T), or rotten(T), ask about the indicated tomato's state.
6. If the fact is at(T, S), ask whether the indicated tomato is attached to or located at the indicated stem/location.
7. The respondent may not recognize identifiers such as stem_01, stem_02, tomato1, or tomato2.
8. Assume the respondent sees only a tablet display where the relevant object or location is visually indicated.
9. Output only the question.

Examples:

Fact: located(brl_robot,stem_01)

Question: 지금 로봇이 화면에 표시된 위치에 있는 것이 맞나요?

Fact: ripe(tomato1)

Question: 화면에 표시된 토마토가 익은 것이 맞나요?

Fact: rotten(tomato2)

Question: 화면에 표시된 토마토가 상한 것이 맞나요?

Fact: at(tomato1,stem_??)

Question: 현재 줄기에 첫번째 토마토가 달려있나요?

Action: {selected action}

Fact: {selected hypothesis}

Predicate meanings:

- observed(T): tomato T has been observed.
- scanned(T): tomato T has been scanned.
- ripe(T): tomato T is ripe.
- unripe(T): tomato T is unripe.
- rotten(T): tomato T is rotten.
- holded(T, R): robot R has held tomato T.
- loaded(T, R): tomato T has been loaded by robot R.
- discarded(T): tomato T has been discarded.
- at(T, S): tomato T is at stem/location S.
- handempty(R): robot R's hand is empty.
- holding(R, T): robot R is holding tomato T.
- located(R, L): robot R is located at location L.

Question:

Answer: 화면에 표시된 토마토가 익은 것이 맞나요?